

bircd 分析

課題添付サンプル（bircd/）の構造分析
目的: ft_irc 実装の骨格を理解する

概要

bircd は**最小限のブロードキャストサーバー**。IRCプロトコルは未実装。
接続したクライアントからのメッセージを、他の全クライアントに転送するだけ。

位置づけ: ソケットプログラミング・I/O多重化の教材

ファイル構成

ファイル	役割
bircd.h	構造体定義、マクロ、関数プロトタイプ
main.c	エントリーポイント
init_env.c	環境構造体の初期化
get_opt.c	コマンドライン引数の解析
srv_create.c	サーバーソケットの作成
srv_accept.c	クライアント接続の受付
main_loop.c	メインイベントループ
init_fd.c	fd_set の初期化
do_select.c	select() の呼び出し
check_fd.c	読み書き可能な fd のチェック
client_read.c	クライアントからのデータ受信
client_write.c	クライアントへのデータ送信
clean_fd.c	fd のクリーンアップ
x.c	エラーハンドリングユーティリティ

データ構造

t_fd（ファイルディスクリプタ情報）

```
typedef struct s_fd {
    int    type;                // FD_FREE, FD_SERV, FD_CLIENT
    void (*fct_read)();         // 読み取り時のコールバック
    void (*fct_write)();        // 書き込み時のコールバック
    char   buf_read[BUF_SIZE + 1]; // 読み取りバッファ
    char   buf_write[BUF_SIZE + 1]; // 書き込みバッファ
} t_fd;
```

t_env（環境/サーバー状態）

```
typedef struct s_env {
    t_fd   *fds;               // fd 配列（インデックス = fd 番号）
    int     port;              // リッスンポート
    int     maxfd;             // 最大 fd 番号
    int     max;               // select() 用の最大値
    int     r;                 // select() の戻り値
    fd_set  fd_read;           // 読み取り監視用 fd_set
    fd_set  fd_write;          // 書き込み監視用 fd_set
} t_env;
```

処理フロー

初期化

```
main()
├─ init_env()      # 環境構造体を初期化
├─ get_opt()       # ポート番号を取得
├─ srv_create()    # サーバソケットを作成
│   └─ getprotobyname("tcp")
│       └─ socket(PF_INET, SOCK_STREAM, ...)
│           └─ bind()
│               └─ listen(s, 42) # backlog = 42
└─ main_loop()     # メインループへ
```

メインループ

```
main_loop() [無限ループ]
├─ init_fd()      # fd_set を初期化、監視対象を設定
├─ do_select()     # select() でイベント待ち
└─ check_fd()     # 各 fd をチェックして処理
    ├─ fct_read()  # 読み取り可能な呼び出し
    └─ fct_write() # 書き込み可能な呼び出し
```

クライアント接続

```
srv_accept() [サーバソケットが読み取り可能時]
├─ accept()       # 新しい接続を受け入れ
└─ fds[cs] 設定
    └─ type = FD_CLIENT
        └─ fct_read = client_read
```

データ処理

```
client_read() [クライアント fd が読み取り可能時]
├─ recv()         # データ受信
├─ r <= 0 の場合
│   └─ close()
│       └─ clean_fd()
└─ r > 0 の場合
    └─ 全クライアントに send() # ブロードキャスト
```

ft_irc との違い

項目	birdcd	ft_irc (実装すべき)
I/O多重化	<code>select()</code>	<code>poll()</code> または同等
言語	C	C++98
プロトコル	なし (単純ブロードキャスト)	IRC プロトコル
認証	なし	PASS コマンド
ユーザー管理	なし	NICK, USER
チャンネル	なし	JOIN, PART, KICK 等
部分データ処理	なし (致命的)	必須 (バッファリング)
エラーハンドリング	最小限	堅牢に

学習ポイント

1. fd 配列の設計

birdcd は `fds[fd番号]` という直接インデックスを使用。
シンプルだが、fd 番号が大きくなると無駄なメモリを消費。

ft_irc での改善案:

- `std::map<int, Client*>` で `fd` → `Client` のマッピング
- または `std::vector<struct pollfd> + std::vector<Client*>`

2. コールバック関数の設計

`bircd` は関数ポインタ (`fct_read` , `fct_write`) を使用。

C++98 では仮想関数やファンクタで代替可能だが、単純な関数ポインタでも可。

3. select() → poll() の変換

```
// bircd (select)
fd_set fd_read;
FD_ZERO(&fd_read);
FD_SET(fd, &fd_read);
select(max + 1, &fd_read, NULL, NULL, NULL);
if (FD_ISSET(fd, &fd_read)) { ... }
```

```
// ft_irc (poll)
struct pollfd fds[MAX_CLIENTS];
fds[i].fd = fd;
fds[i].events = POLLIN;
poll(fds, nfd, -1);
if (fds[i].revents & POLLIN) { ... }
```

4. 部分データ処理の欠如

`bircd` は `recv()` で受信したデータをそのまま転送。

IRC では `\r\n` でメッセージが区切られるため、バッファリングが必須。

```
// ft_irc で必要な処理
class Client {
    std::string recv_buffer; // 受信バッファ

    void appendData(const char* data, size_t len) {
        recv_buffer.append(data, len);
    }

    bool hasCompleteMessage() {
        return recv_buffer.find("\r\n") != std::string::npos;
    }

    std::string extractMessage() {
        size_t pos = recv_buffer.find("\r\n");
        std::string msg = recv_buffer.substr(0, pos);
        recv_buffer.erase(0, pos + 2);
        return msg;
    }
};
```

実行方法

```
cd bircd
make
./bircd 6667

# 別ターミナルで
nc localhost 6667
```

チェックリスト

`bircd` を理解したら、以下を確認：

- ☐ `socket()` , `bind()` , `listen()` , `accept()` の役割を説明できる
- ☐ `select()` の引数と戻り値を理解している
- ☐ `fd_set` の操作 (`FD_ZERO` , `FD_SET` , `FD_ISSET`) を理解している
- ☐ なぜ `select()` の代わりに `poll()` を使うか説明できる
- ☐ 部分データ処理がなぜ必要か説明できる